Diagramas de estados del sistema

Proyecto TuEvento: análisis del software, vistas dinámicas

Jhampier Santos Ortiz y Cristofer David Lozano Contreras

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software (ADSO), Ficha 3145556

Instructora: Carlos Julio Cadena

Octubre de 2026

Versión 1.0

Introducción y alcance

Este documento describe el comportamiento dinámico de las entidades principales de TuEvento mediante diagramas de estados del Lenguaje Unificado de Modelado (UML) (Object Management Group [OMG], 2017). Cada diagrama y cada tabla de transiciones se construyó a partir de la lectura del código fuente del backend, desarrollado con Spring Boot bajo arquitectura hexagonal (Cockburn, 2005), y del servicio auxiliar fake-payment-gateway. Solo se documentan las transiciones que existen en el código; no se agregaron transiciones lógicas que no estén implementadas.

Cada sección presenta el enum o clase de estado, el estado inicial y los finales, el diagrama de estados, la tabla de transiciones con su disparador, las condiciones de guarda (guards) y la clase y el método donde ocurre, y las observaciones sobre inconsistencias detectadas. Al final se documentan las relaciones entre entidades, los procesos automáticos, las diferencias con la documentación existente y los puntos abiertos.

En los diagramas, el círculo inicial indica el estado inicial y el círculo final indica que la entidad no tiene más transiciones; las notas señalan particularidades del código. Las etiquetas de cada flecha indican el evento que dispara la transición. Salvo donde se indique lo contrario, todos los valores de cada enum se usan en el código. Los hallazgos se contrastaron con el repositorio del proyecto en la rama develop y en las 12 ramas remotas existentes.

Resumen por entidad

La Tabla 1 resume las diez máquinas de estados analizadas y el hallazgo principal de cada una.

Tabla 1

Resumen de las máquinas de estados del sistema

Entidad	N.º de estados	Estado inicial	Estados finales	Hallazgo principal
Evento	6	DRAFT	CANCELLED, COMPLETED	Flujo de revisión admin completo; las reglas viven en dos switch, sin allowedTransitions()
Orden	6	DRAFT	CANCELLED, REFUNDED, USED	USED nunca se asigna; la orden queda en PAID tras el check-in
Ticket	6	PENDING	REFUNDED, USED, CANCELLED	Sin guardas de transición: asigna el estado directamente
Asiento	4	AVAILABLE	SOLD	COURTESY sin uso; updateSeatStatus() acepta cualquier combinación
Pago (backend)	5	PENDING o APPROVED	REJECTED, ERROR, REFUNDED	Dos estados iniciales; CANCELLED del gateway se mapea a ERROR
Pago (gateway)	7	PENDING	REFUNDED, DECLINED, FAILED, CANCELLED	PROCESSING es transitorio; el backend ignora su webhook
Billetera (transacción)	3	PENDING o COMPLETED	COMPLETED, FAILED	Dos estados iniciales según el tipo de transacción
Notificación (entrega)	4	PENDING, DELIVERED o FAILED	DELIVERED, SENT, FAILED	SENT → DELIVERED no se implementa; leer no cambia el estado de entrega
Cuenta de usuario	5	PENDING	(ninguno real)	DELETED no se asigna en ningún caso de uso; reactivación sin documentar
Reembolso	4	REQUESTED	PROCESSED, FAILED	Solo REQUESTED → PROCESSED está implementada

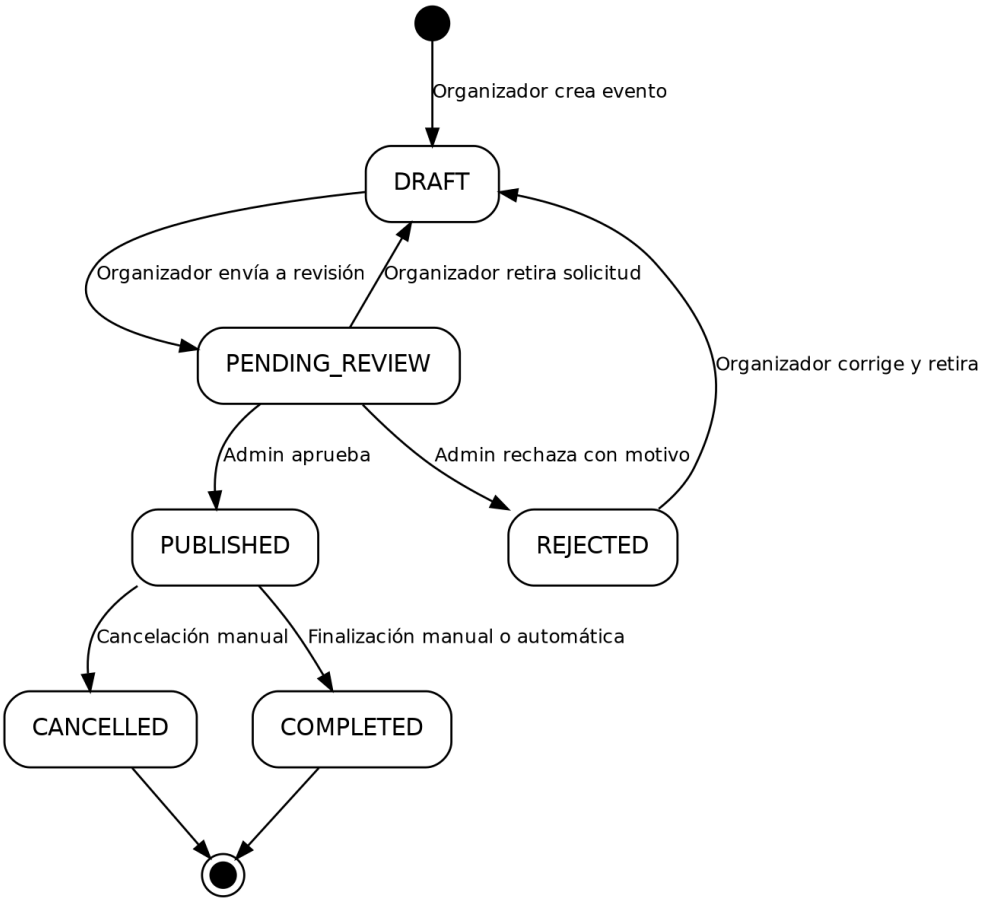
Diagramas de estados por entidad

1. Evento

El estado del evento se define en el enum EventStatus (modules/event/domain/model/EventStatus.java) y toma los valores DRAFT, PENDING_REVIEW, PUBLISHED, REJECTED, CANCELLED y COMPLETED. El estado inicial es DRAFT, asignado en CreateEventService.execute(). CANCELLED y COMPLETED son terminales de hecho, porque ningún caso de uso permite salir de ellos. La Figura 1 muestra el ciclo de vida completo, incluido el flujo de revisión del administrador; la flecha PUBLISHED → COMPLETED agrupa la finalización manual y la automática.

Figura 1

Diagrama de estados de la entidad Evento



La Tabla 2 detalla cada transición con sus guardas y la clase responsable.

Tabla 2

Transiciones de la entidad Evento

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	DRAFT	Organizador crea el evento	Fechas válidas (no pasadas, no muy lejanas, fin ≥ inicio); el sitio existe; availableSeats ≤ capacidad del sitio; nombre + fecha + sitio únicos; categoría activa	CreateEventService.execute()
DRAFT	PENDING_REVIEW	Organizador envía a revisión	startDate ≥ hoy; entre 3 y 9 imágenes; al menos 1 sección configurada; el userId del JWT es el dueño	ChangeEventStatusService.execute()
PENDING_REVIEW	DRAFT	Organizador retira la solicitud	El userId del JWT es el dueño; se limpia rejectionReason	ChangeEventStatusService.execute()
REJECTED	DRAFT	Organizador corrige y retira para editar	El userId del JWT es el dueño; se limpia rejectionReason	ChangeEventStatusService.execute()
PENDING_REVIEW	PUBLISHED	Admin aprueba	startDate ≥ hoy; entre 3 y 9 imágenes; al menos 1 sección; sin verificación de ownership	AdminChangeEventStatusUseCase.execute()
PENDING_REVIEW	REJECTED	Admin rechaza	reason no nulo ni vacío; se persiste en event.rejectionReason	AdminChangeEventStatusUseCase.execute()
PUBLISHED	CANCELLED	Cancelación manual	Sin validaciones adicionales	ChangeEventStatusService.execute() (organizador); AdminChangeEventStatusUseCase.execute() (admin)
PUBLISHED	COMPLETED	Finalización manual	LocalDate.now().isAfter(event.getFinishDate()): solo después del día de fin	ChangeEventStatusService.execute(); AdminChangeEventStatusUseCase.execute()
PUBLISHED	COMPLETED	Finalización automática	finishDate < hoy; cron 0 5 0 * * * y ejecución de recuperación en ApplicationReadyEvent; changedBy = null	EventAutoCompletionScheduler.completeExpiredEvents()

Se verificó que no existe la transición DRAFT → CANCELLED (un borrador no se puede cancelar directamente) ni REJECTED → PENDING_REVIEW (el organizador debe pasar primero por DRAFT). Además, el enum no define allowedTransitions(): las reglas están en el método validateTransition() de los dos casos de uso, por lo que agregar un tercero sin actualizar ambos dejaría la máquina de estados inconsistente.

2. Orden

La orden usa el enum OrderStatus (modules/ticket/domain/model/OrderStatus.java), que sí implementa allowedTransitions(), canTransitionTo() e isTerminal(). Sus valores son DRAFT, PAYMENT_PENDING, PAID, CANCELLED, REFUNDED y USED. El estado inicial es DRAFT, asignado en CreateOrderWithTicketsUseCaseImpl.execute(), y los finales declarados son CANCELLED, REFUNDED y USED. La Figura 2 incluye la transición PAID → USED porque el enum la declara, aunque ningún caso de uso la ejecuta.

Figura 2

Diagrama de estados de la entidad Orden

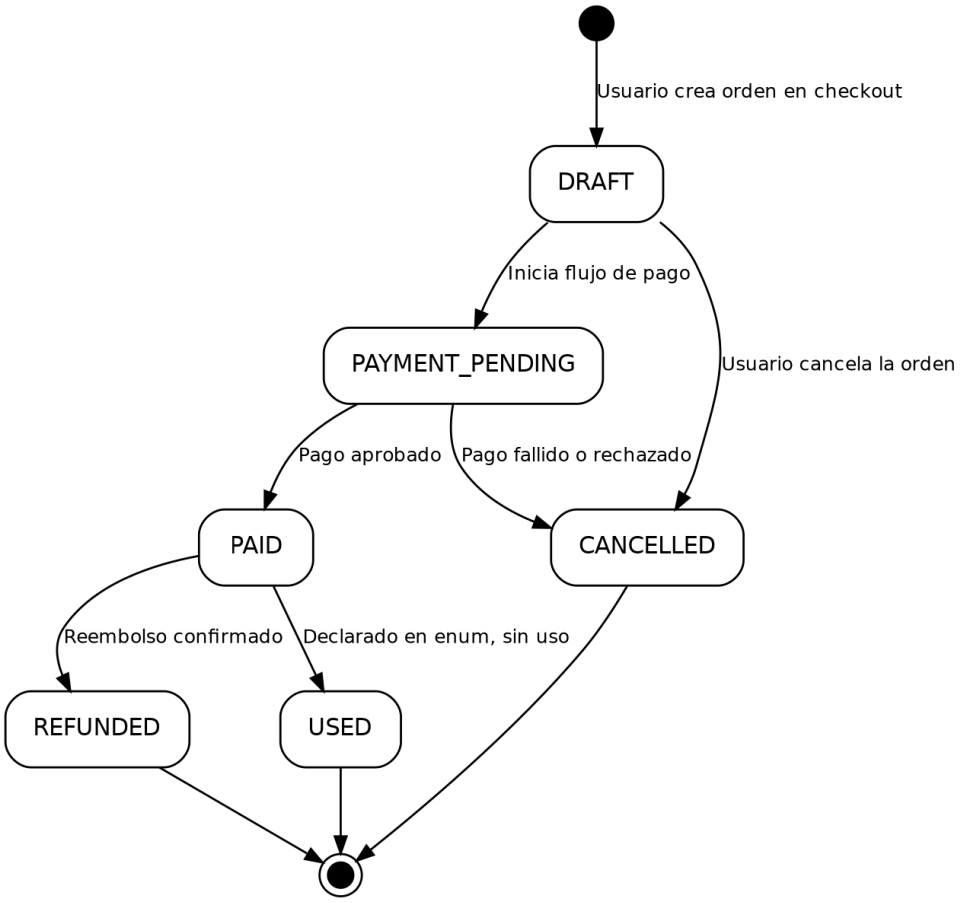


Tabla 3

Transiciones de la entidad Orden

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	DRAFT	Usuario crea la orden en el checkout	Todas las sillas en RESERVED por ese userId; reservas no expiradas (reservedUntil > now)	CreateOrderWithTicketsUseCaseImpl.execute()
DRAFT	PAYMENT_PENDING	Usuario inicia el flujo de pago	canTransitionTo(); invocado por el orquestador de pagos	InitiateOrderPaymentUseCaseImpl.initiateOrderPayment() → order.markAsPaymentPending()
DRAFT	CANCELLED	Usuario cancela la orden	canTransitionTo(); las sillas se liberan a AVAILABLE	CancelOrderUseCaseImpl.execute() → order.cancel()
PAYMENT_PENDING	PAID	Pago aprobado (webhook APPROVED)	canTransitionTo()	ConfirmOrderPaymentUseCaseImpl.confirmOrderPayment() → order.markAsPaid()
PAYMENT_PENDING	CANCELLED	Pago fallido, rechazado o cancelado (webhooks DECLINED, FAILED, CANCELLED)	canTransitionTo(); las sillas se liberan a AVAILABLE	FailOrderPaymentUseCaseImpl.failOrderPayment() → order.cancel()
PAID	REFUNDED	Reembolso confirmado (webhook REFUNDED o pago solo con wallet)	canTransitionTo()	ConfirmRefundUseCaseImpl.confirmRefund() → order.refund()

El estado USED nunca se usa: el método Order.markAsUsed() existe (Order.java, línea 70), pero no tiene ninguna llamada en el proyecto. El check-in solo cambia el Ticket a USED (CheckinTicketUseCaseImpl, línea 55); la orden permanece en PAID indefinidamente. Por otra parte, el webhook CANCELLED del gateway marca el Payment como ERROR (markAsError()), pero cancela la orden mediante failOrderPayment() → order.cancel().

3. Ticket

El ticket usa el enum simple TicketStatus (modules/ticket/domain/model/TicketStatus.java), sin allowedTransitions(). Sus valores son PENDING, PROCESSING, PAID, REFUNDED, USED y CANCELLED. El estado inicial es PENDING, asignado en CreateOrderWithTicketsUseCaseImpl.execute(); los finales son REFUNDED, USED y CANCELLED, porque ningún caso de uso los lee como origen. La Figura 3 muestra su ciclo de vida.

Figura 3

Diagrama de estados de la entidad Ticket

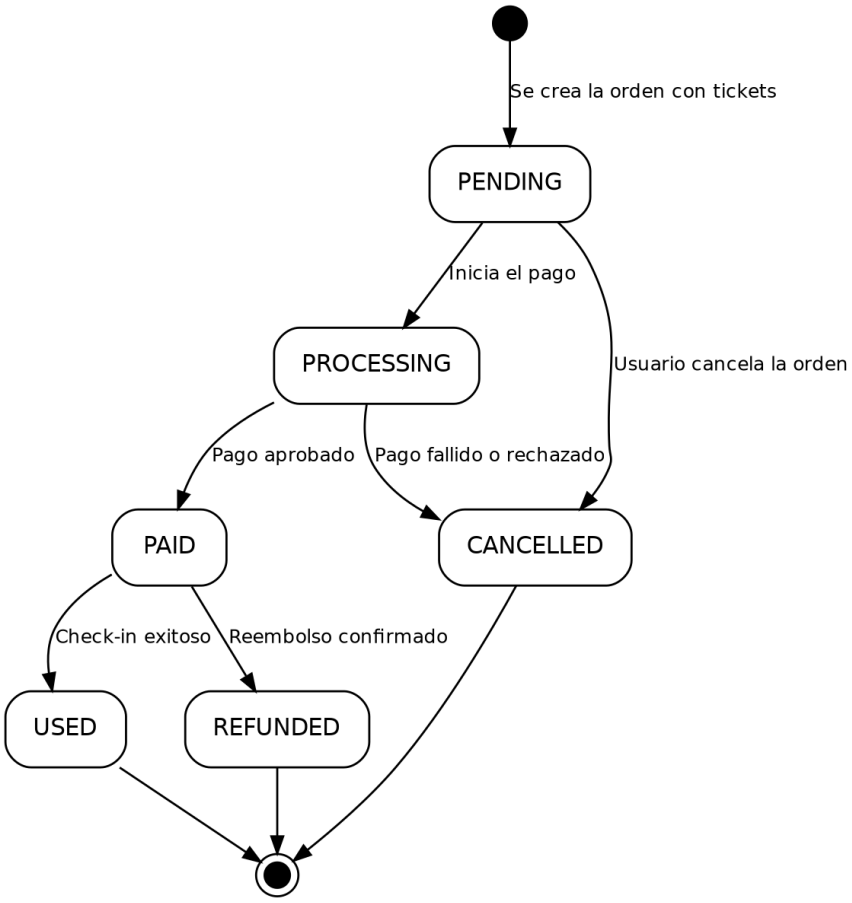


Tabla 4

Transiciones de la entidad Ticket

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	PENDING	Se crea la orden con tickets	Sillas RESERVED por el mismo usuario y reservas vigentes	CreateOrderWithTicketsUseCaseImpl.execute()
PENDING	PROCESSING	Inicio del flujo de pago	Sin guardia de estado origen (se asigna vía builder)	InitiateOrderPaymentUseCaseImpl.initiateOrderPayment()
PROCESSING	PAID	Pago aprobado (webhook APPROVED)	Sin guardia; ticket.markAsPaid() asigna directo	ConfirmOrderPaymentUseCaseImpl.confirmOrderPayment() → ticket.markAsPaid()
PROCESSING	CANCELLED	Pago fallido, rechazado o cancelado	Sin guardia; ticket.cancel() asigna directo	FailOrderPaymentUseCaseImpl.failOrderPayment() → ticket.cancel()
PAID	USED	Check-in completado	ticket.canCheckin(): status == PAID, expirationDate != null y now < expirationDate; no debe existir un check-in previo en TicketCheckinRepository	CheckinTicketUseCaseImpl.execute() → ticket.markAsUsed()
PAID	REFUNDED	Reembolso confirmado (webhook REFUNDED o pago solo con wallet)	Sin guardia de estado; ticket.markAsRefunded() asigna directo	ConfirmRefundUseCaseImpl.confirmRefund() → ticket.markAsRefunded()
PENDING	CANCELLED	El usuario cancela la orden directamente	Sin guardia; ticket.cancel() asigna directo; las sillas se liberan	CancelOrderUseCaseImpl.execute() → ticket.cancel()

A diferencia de la orden y el pago, el ticket no tiene guardas de transición: los métodos markAsPaid(), markAsUsed(), markAsRefunded() y cancel() asignan this.status directamente sin verificar el estado anterior (Ticket.java), por lo que una llamada accidental podría producir CANCELLED → PAID sin que el código la rechace. Asimismo, PENDING → CANCELLED ocurre en CancelOrderUseCaseImpl aunque el flujo natural sería PROCESSING → CANCELLED.

4. Asiento

El asiento usa el enum simple SeatStatus (modules/seat/domain/model/SeatStatus.java), sin allowedTransitions(), con los valores AVAILABLE, RESERVED, SOLD y COURTESY. El estado inicial es AVAILABLE, asignado en SeatService.createSeat(). SOLD es el único estado final de hecho: las sillas vendidas no se liberan tras un reembolso, decisión de negocio documentada en ConfirmRefundUseCaseImpl; sin embargo, un administrador sí puede cambiar manualmente cualquier estado mediante SeatService.updateSeatStatus(). La Figura 4 dibuja cada causa de liberación por separado y no incluye el cambio manual del administrador.

Figura 4

Diagrama de estados de la entidad Asiento

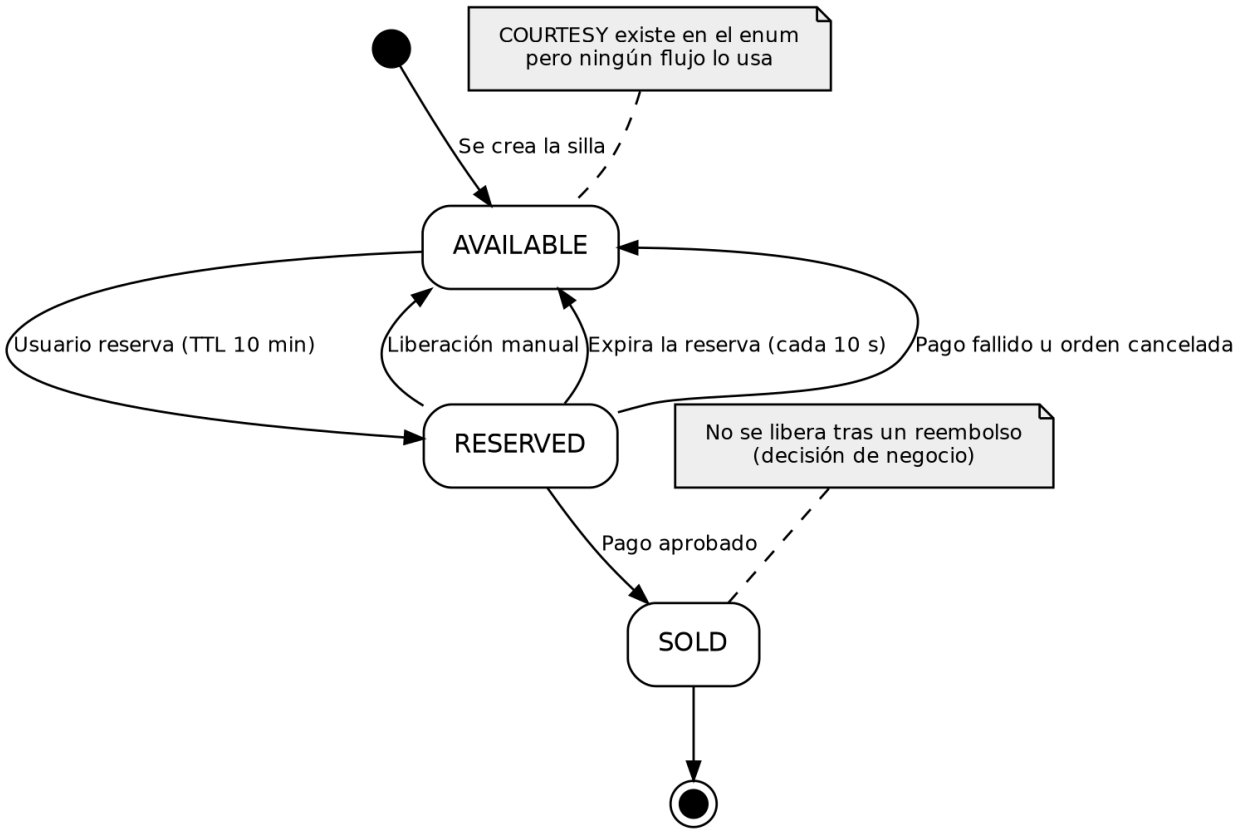


Tabla 5

Transiciones de la entidad Asiento

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	AVAILABLE	Creación de la silla en la sección	Sin validación extra; implícito	SeatService.createSeat()
AVAILABLE	RESERVED	Usuario reserva la silla (TTL de 10 min)	seat.status == AVAILABLE; reservedUntil = now + 10 min; lanza SEAT_NOT_AVAILABLE si no está AVAILABLE	SeatReservationService.reserveSeat()
RESERVED	AVAILABLE	Usuario libera manualmente la reserva	seat.status == RESERVED; el userId debe coincidir con reservedBy, salvo que isAdmin = true	SeatReservationService.releaseSeat()
RESERVED	AVAILABLE	Expiración automática del TTL (cada 10 s)	seat.status == RESERVED y reservedUntil < now; cron */10 * * * * * y ejecución en ApplicationReadyEvent	SeatReservationExpirationScheduler.releaseExpiredReservations()
RESERVED	SOLD	Pago aprobado (webhook APPROVED)	Sin guardia de estado origen; solo valida que el nuevo estado sea distinto al actual (SEAT_STATUS_UNCHANGED)	ConfirmOrderPaymentUseCaseImpl.confirmOrderPayment() → seatUseCase.updateSeatStatus(seatId, SOLD, null)
RESERVED	AVAILABLE	Pago fallido, rechazado o cancelado	Sin guardia de estado origen; solo valida que el nuevo estado sea distinto	FailOrderPaymentUseCaseImpl.failOrderPayment() → seatUseCase.updateSeatStatus(seatId, AVAILABLE, reason)
RESERVED	AVAILABLE	Cancelación de la orden por el usuario	Sin guardia de estado origen	CancelOrderUseCaseImpl.execute() → seatUseCase.updateSeatStatus(seatId, AVAILABLE, "Order cancelled")
Cualquiera	Cualquiera	Cambio manual por un admin	Solo valida que el nuevo estado sea distinto al actual	SeatService.updateSeatStatus() (SeatController)

COURTESY está en el enum SeatStatus, pero ningún caso de uso lo asigna ni lo verifica (un administrador podría enviarlo por REST). No debe confundirse con SeatType.COURTESY, un enum distinto (REGULAR, COURTESY) que clasifica el tipo de silla y no su estado. Por otra parte, updateSeatStatus() no valida el estado origen y permite, por ejemplo, SOLD → AVAILABLE si un administrador lo solicita; y el scheduler libera reservas cada 10 s sin generar SeatLog, mientras que SeatService.updateSeatStatus() sí lo genera.

5. Pago (backend)

El pago del backend usa el enum PaymentStatus (modules/payment/domain/model/PaymentStatus.java), que implementa allowedTransitions(), canTransitionTo() e isTerminal(). Sus valores son PENDING, APPROVED, REJECTED, ERROR y REFUNDED. Tiene dos estados iniciales: PENDING, asignado en ChargeOrderViaGatewayUseCase.execute() (línea 77) para pagos con pasarela, totales o mixtos; y APPROVED, asignado en CreateWalletOnlyPaymentUseCase.execute() (línea 60) para pagos pagados íntegramente con wallet, sin pasar por PENDING. REJECTED, ERROR y REFUNDED están declarados como terminales. La Figura 5 muestra ambos orígenes.

Figura 5

Diagrama de estados de la entidad Pago (backend)

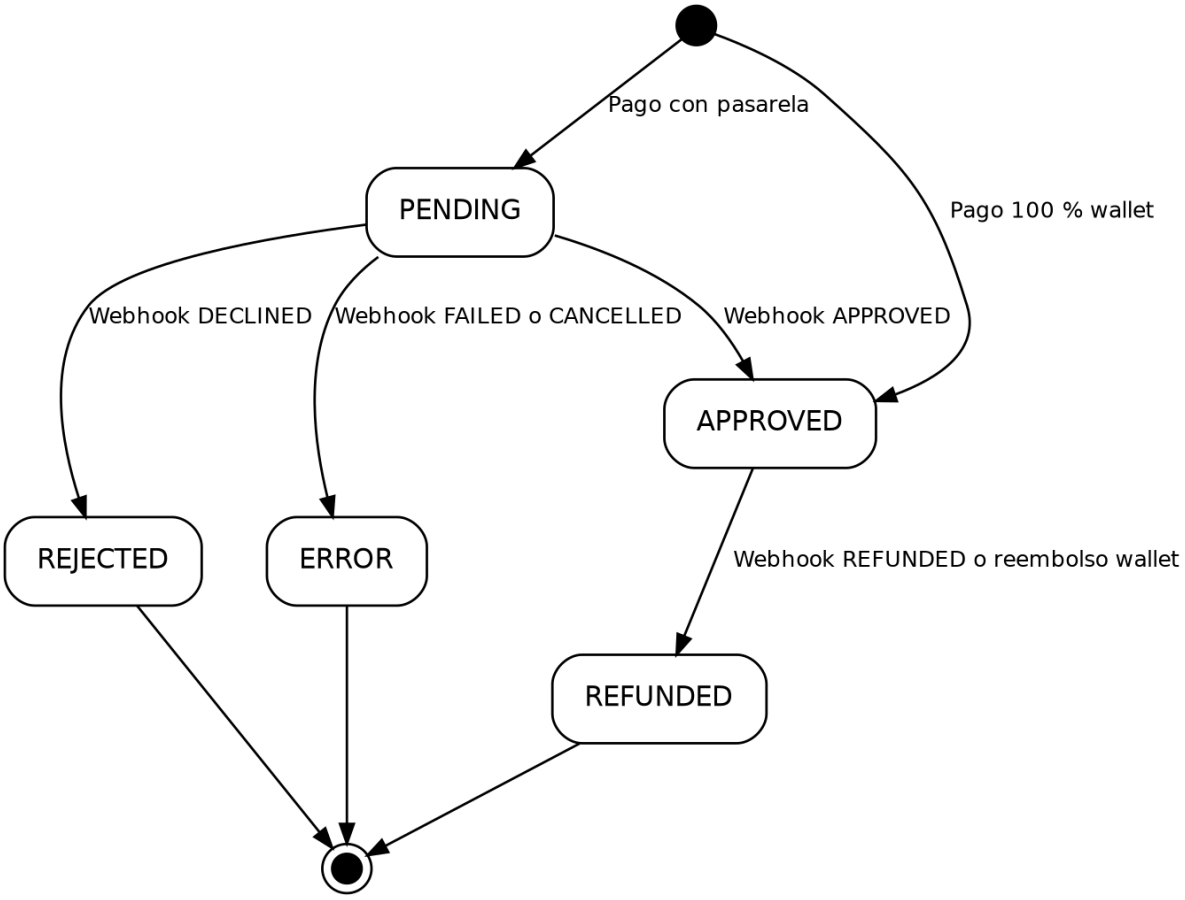


Tabla 6

Transiciones de la entidad Pago (backend)

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	PENDING	Creación de pago con pasarela (total o mixto)	Orden en PAYMENT_PENDING; invocado por CreatePaymentUseCaseImpl	ChargeOrderViaGatewayUseCase.execute()
(nuevo)	APPROVED	Creación de pago 100 % wallet	Saldo disponible ≥ total de la orden; sin pasarela	CreateWalletOnlyPaymentUseCase.execute()
PENDING	APPROVED	Webhook APPROVED del gateway	Firma HMAC válida; idempotencia por gatewayEventId; canTransitionTo(APPROVED)	ProcessWebhookUseCaseImpl.execute() → payment.approve()
PENDING	REJECTED	Webhook DECLINED	Mismas verificaciones de firma e idempotencia; canTransitionTo(REJECTED)	ProcessWebhookUseCaseImpl.execute() → payment.reject()
PENDING	ERROR	Webhook FAILED	Mismas verificaciones; canTransitionTo(ERROR)	ProcessWebhookUseCaseImpl.execute() → payment.markAsError()
PENDING	ERROR	Webhook CANCELLED	Mismas verificaciones; el CANCELLED del gateway se mapea a ERROR (mismo markAsError())	ProcessWebhookUseCaseImpl.execute() → payment.markAsError()
APPROVED	REFUNDED	Webhook REFUNDED	Firma HMAC válida; canTransitionTo(REFUNDED)	ProcessWebhookUseCaseImpl.execute() → payment.refund()
APPROVED	REFUNDED	Reembolso de un pago solo con wallet	payment.status == APPROVED (si no, lanza IllegalStateException); canTransitionTo(REFUNDED)	RequestRefundUseCaseImpl.closeRefundLocally() → payment.refund()

El webhook CANCELLED de la pasarela lleva el Payment a ERROR, porque este enum no tiene el valor CANCELLED. Es intencional, pero puede confundir en el registro de auditoría. Además, CreateWalletOnlyPaymentUseCase registra en PaymentLog un cambio PENDING → APPROVED que nunca existió en la base de datos; es un registro conceptual para trazabilidad.

6. Pago (pasarela simulada, fake-payment-gateway)

El servicio auxiliar fake-payment-gateway simula la pasarela de pagos. Su enum PaymentStatus (fake-payment-gateway/.../domain/model/PaymentStatus.java) implementa allowedTransitions(), canTransitionTo() e isTerminal(), y define PENDING, PROCESSING, APPROVED, REFUNDED, DECLINED, FAILED y CANCELLED. El estado inicial es PENDING, asignado en CreatePaymentUseCase.execute() → Payment.create(); los terminales son REFUNDED, DECLINED, FAILED y CANCELLED. Tras cada transición, WebhookNotifierPort.notify(PaymentStatusChanged) envía un evento HTTP al backend. La Figura 6 muestra el flujo.

Figura 6

Diagrama de estados del Pago en la pasarela simulada

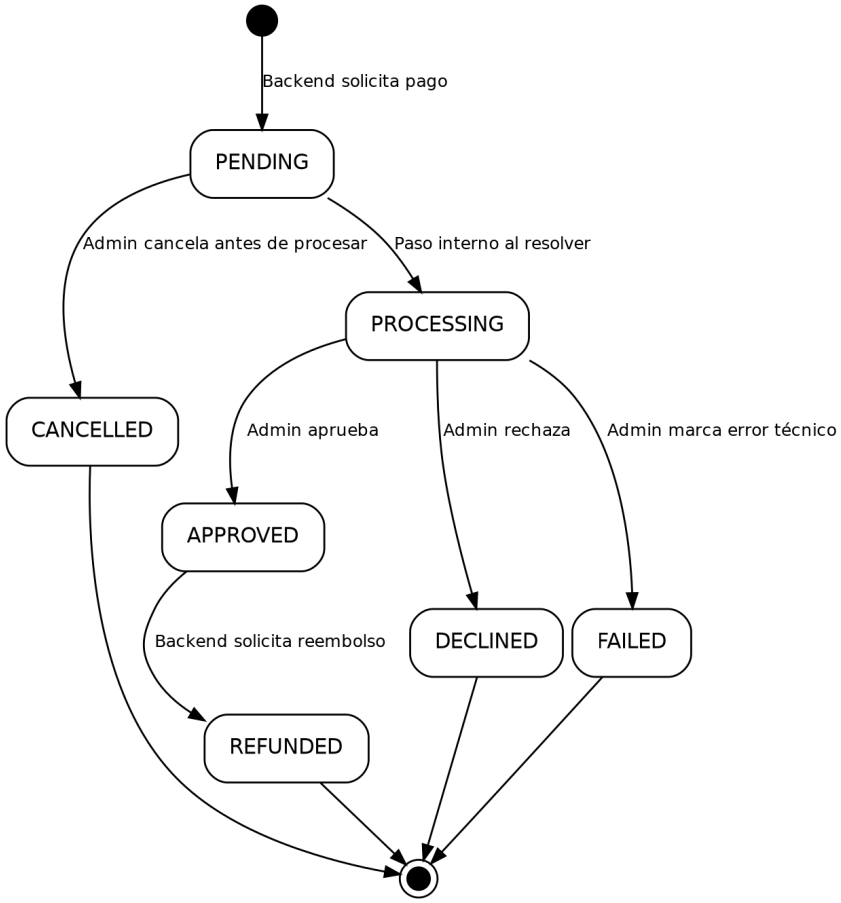


Tabla 7

Transiciones del Pago en la pasarela simulada

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	PENDING	Solicitud de creación desde el backend	Sin validaciones adicionales	CreatePaymentUseCase.execute() → Payment.create()
PENDING	PROCESSING	Paso intermedio interno al aprobar, declinar o fallar	payment.canTransitionTo(PROCESSING)	ApprovePaymentUseCase, DeclinePaymentUseCase, FailPaymentUseCase → payment.markAsProcessing()
PENDING	CANCELLED	El admin cancela antes de procesar	payment.canTransitionTo(CANCELLED)	CancelPaymentUseCase.execute() → payment.cancel()
PROCESSING	APPROVED	El admin aprueba	payment.canTransitionTo(APPROVED)	ApprovePaymentUseCase.execute() → payment.approve()
PROCESSING	DECLINED	El admin rechaza	payment.canTransitionTo(DECLINED)	DeclinePaymentUseCase.execute() → payment.decline()
PROCESSING	FAILED	El admin marca error técnico	payment.canTransitionTo(FAILED)	FailPaymentUseCase.execute() → payment.fail()
APPROVED	REFUNDED	El backend solicita el reembolso	payment.canTransitionTo(REFUNDED)	RefundPaymentUseCase.execute() → payment.refund()

PROCESSING es un estado transitorio interno: el gateway pasa de PENDING a PROCESSING y luego a APPROVED, DECLINED o FAILED dentro de la misma llamada a execute(), enviando dos webhooks. ProcessWebhookUseCaseImpl (backend) no tiene un case para PROCESSING; ese valor cae en el default y solo registra un log.warn. Se verificó que esto es inofensivo: el gateway siempre envía PROCESSING antes que APPROVED, y si llegara un evento repetido la idempotencia por gatewayEventId lo descartaría.

7. Billetera (transacción de wallet)

Este diagrama representa el estado de cada transacción de la billetera (WalletTransaction), no el saldo de la billetera. Usa el enum simple WalletTransactionStatus (modules/wallet/domain/model/WalletTransactionStatus.java), con los valores PENDING, COMPLETED y FAILED. El estado inicial depende del tipo de transacción (WalletTransactionType): PENDING para PAYMENT, asignado en ReserveWalletPaymentUseCaseImpl.execute(), y COMPLETED para CREDIT, REVERSAL y ADJUSTMENT, que se crean directamente. COMPLETED y FAILED son terminales. La Figura 7 muestra los dos orígenes.

Figura 7

Diagrama de estados de la transacción de wallet

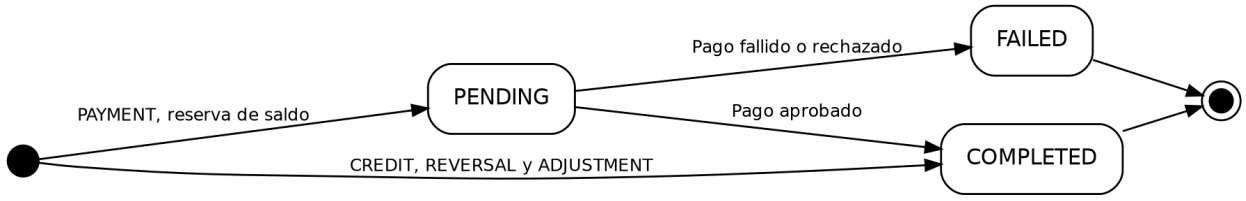


Tabla 8

Transiciones de la transacción de wallet

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	PENDING	Reserva de saldo para un pago (checkout)	saldoDisponible = balance − sumPendingPayments ≥ amount; bloqueo optimista @Version en Wallet	ReserveWalletPaymentUseCaseImpl.execute()
(nuevo)	COMPLETED	Crédito directo (cancelación de evento, ajuste admin)	Idempotencia por idempotencyKey; sin otra validación de estado origen	CreditWalletUseCaseImpl.execute()
(nuevo)	COMPLETED	Reversión de un PAYMENT (reembolso con wallet)	Idempotencia por reversal-{originalTransactionId}; la transacción original debe estar en COMPLETED	ReverseWalletPaymentUseCaseImpl.execute()
(nuevo)	COMPLETED	Ajuste manual de balance (admin)	amount != 0; reason no vacío; balance resultante ≥ 0	AdjustWalletUseCaseImpl.execute()
PENDING	COMPLETED	Pago aprobado (webhook APPROVED o pago solo con wallet)	Idempotencia: si ya está COMPLETED no hace nada; descuenta el balance del Wallet	ConfirmWalletPaymentUseCaseImpl.execute() → tx.complete(newBalance)
PENDING	FAILED	Pago fallido, rechazado o cancelado (webhooks DECLINED, FAILED, CANCELLED)	Idempotencia: si ya está FAILED no hace nada; no toca el balance	ReleaseWalletPaymentUseCaseImpl.execute() → tx.fail()

Las transacciones CREDIT, REVERSAL y ADJUSTMENT nacen directamente en COMPLETED sin pasar por PENDING; por eso el diagrama tiene dos estados iniciales. COMPLETED y FAILED son terminales: no existe ninguna transición de salida.

8. Notificación (entrega a cada destinatario)

El enum DeliveredStatus (modules/notification/domain/model/DeliveredStatus.java) aplica a la entidad NotificationUser, que relaciona una Notification con cada destinatario; la Notification principal no tiene estado propio. Sus valores son PENDING, SENT, DELIVERED y FAILED. El estado inicial depende del canal: IN_APP nace en DELIVERED, EMAIL nace en PENDING, y si hay un error antes de crear el registro (correo nulo o excepción al guardar) nace directamente en FAILED. DELIVERED, SENT y FAILED son finales, porque ningún caso de uso los lee como origen. La Figura 8 muestra los tres orígenes posibles.

Figura 8

Diagrama de estados de la entrega de notificaciones

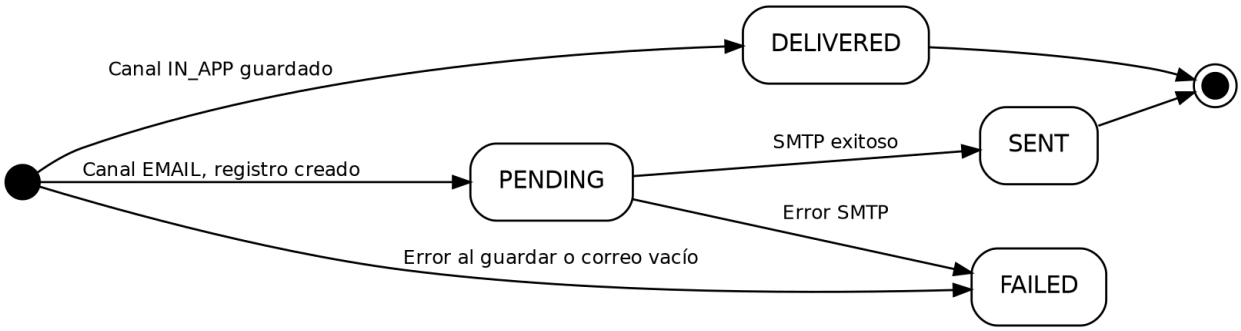


Tabla 9

Transiciones de la entrega de notificaciones

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	DELIVERED	Canal IN_APP: guardar en BD equivale a entregar	Intento adicional de push por WebSocket (si falla, no bloquea ni cambia el estado)	InAppChannelAdapter.deliver() → deliveredStatus(DELIVERED)
(nuevo)	FAILED	Canal IN_APP: excepción al guardar en BD	Bloque catch externo del for; intenta guardar con FAILED	InAppChannelAdapter.deliver() → catch
(nuevo)	FAILED	Canal EMAIL: usuario sin correo o correo en blanco	loginCredentialsRepository.findByUserId(userId) vacío o correo vacío	EmailChannelAdapter.deliver() (guarda FAILED sin crear PENDING previo)
(nuevo)	PENDING	Canal EMAIL: registro creado antes de enviar	Correo del usuario encontrado y no vacío	EmailChannelAdapter.deliver() → deliveredStatus(PENDING)
PENDING	SENT	Correo enviado por SMTP	mailSender.send() sin excepción	EmailChannelAdapter.deliver() (actualiza a SENT)
PENDING	FAILED	Error de SMTP o MessagingException	Bloque catch; actualiza el registro PENDING existente	EmailChannelAdapter.deliver() → catch (actualiza a FAILED)

No existe la transición SENT → DELIVERED (el enum la sugiere implícitamente, pero ningún código la implementa) ni PENDING → DELIVERED directa. DELIVERED (IN_APP) y SENT (EMAIL) son ambos terminales en la práctica, aunque semánticamente distintos. Además, NotificationUser tiene el campo readAt y un endpoint para marcar como leída, pero esa acción solo actualiza readAt (InAppNotificationQueryUseCase) y no cambia deliveredStatus: leer y entregar son conceptos independientes en este código.

9. Cuenta de usuario

El estado de la cuenta no es un enum Java: UserStatus (modules/security/domain/model/UserStatus.java) es una entidad de la tabla user_status, manipulada como String con el campo code, cuyos valores se insertan en la migración Liquibase 017. Los valores son PENDING, ACTIVE, BLOCKED, INACTIVE y DELETED. El estado inicial es PENDING, asignado en RegisterUserUseCase.register() (DEFAULT_STATUS_CODE). En teoría DELETED sería el final, pero ningún caso de uso lo asigna, así que ningún estado es realmente terminal. La Figura 9 muestra el ciclo de vida.

Figura 9

Diagrama de estados de la Cuenta de usuario

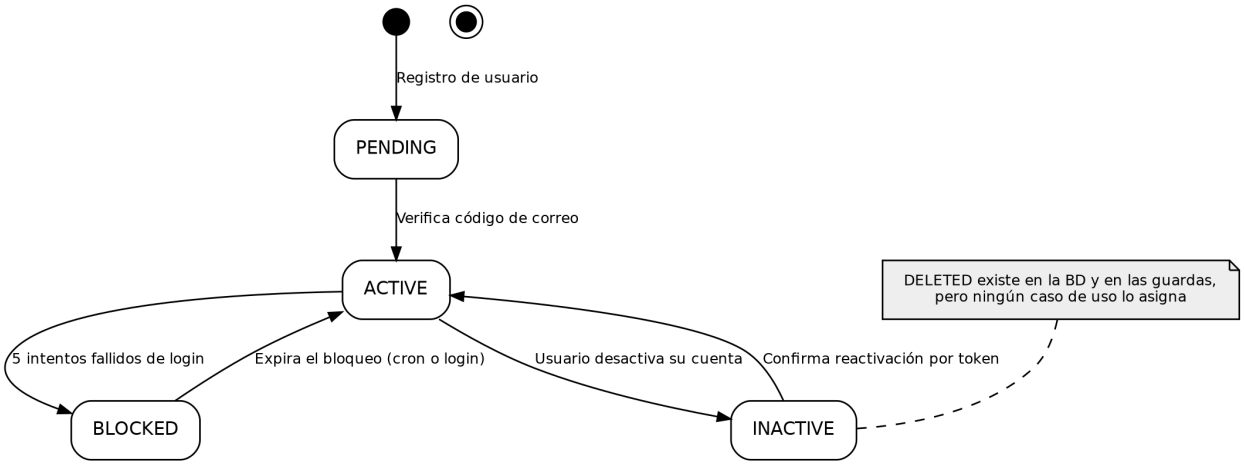


Tabla 10

Transiciones de la Cuenta de usuario

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	PENDING	Registro de nuevo usuario	Email único y no registrado; @gmail.com obligatorio; contraseña fuerte; fullName válido	RegisterUserUseCase.register()
PENDING	ACTIVE	Verifica su correo con el código de activación	Código correcto, no expirado (expiresAt > now), no usado y validado contra el userId	ActivateAccountUseCase.activate() → user.setUserStatus(ACTIVE)
ACTIVE	BLOCKED	5 intentos fallidos de login en una ventana de 15 min	failedAttempts ≥ MAX_FAILED_ATTEMPTS (5); lockedUntil = now + LOCKOUT_MINUTES; transacción independiente (REQUIRES_NEW)	AccountLockoutService.saveLockout() → blockUser()
BLOCKED	ACTIVE	Expiración del bloqueo por tiempo (cron cada minuto)	lockedUntil < now; transacción independiente (REQUIRES_NEW)	SecurityDataCleanupTask.unlockExpiredLockouts() → deleteLockout() → unblockUser()
BLOCKED	ACTIVE	Login después de que el bloqueo expiró	lockedUntil < now detectado durante el login; se elimina el registro AccountLockout	LoginUseCase.checkLockout() → deleteLockout() → unblockUser()
ACTIVE	INACTIVE	El propio usuario desactiva su cuenta	Contraseña verificada; cuenta no INACTIVE ni DELETED; se revocan sesiones y refresh tokens activos	DeactivateAccountUseCase.deactivate() → user.setUserStatus(INACTIVE)
INACTIVE	ACTIVE	Confirma la reactivación con el token recibido por correo	Token válido, no usado y no expirado (30 min); el token se invalida tras su uso	ConfirmReactivationUseCase.confirm() → user.setUserStatus(ACTIVE)

Se verificó con git grep en las 12 ramas remotas que DELETED solo aparece como guarda en LoginUseCase y DeactivateAccountUseCase; ningún caso de uso lo asigna, por lo que la eliminación de cuenta del RF7 del SRS no está implementada. El flujo de reactivación (RequestReactivationUseCase y ConfirmReactivationUseCase) no está documentado en el glosario ni en el manual técnico anterior. El bloqueo usa además la entidad AccountLockout (contador de intentos y expiración); el estado vuelve a ACTIVE al eliminar ese registro, como efecto de deleteLockout() y no como una transición declarada en un enum. No existe BLOCKED → INACTIVE ni BLOCKED → DELETED: el usuario bloqueado se desbloquea solo y vuelve a ACTIVE.

10. Reembolso

El reembolso usa el enum RefundStatus (modules/payment/domain/model/RefundStatus.java), con los valores REQUESTED, APPROVED, PROCESSED y FAILED. El propio enum indica que es una estructura lista para una fase futura. El estado inicial es REQUESTED, asignado en RequestRefundUseCaseImpl (línea 70), y los finales son PROCESSED y FAILED. La entidad Refund es un objeto inmutable construido con builder y no tiene métodos de transición: cada cambio de estado crea un objeto nuevo. La Figura 10 muestra que solo la transición REQUESTED → PROCESSED está implementada.

Figura 10

Diagrama de estados de la entidad Reembolso

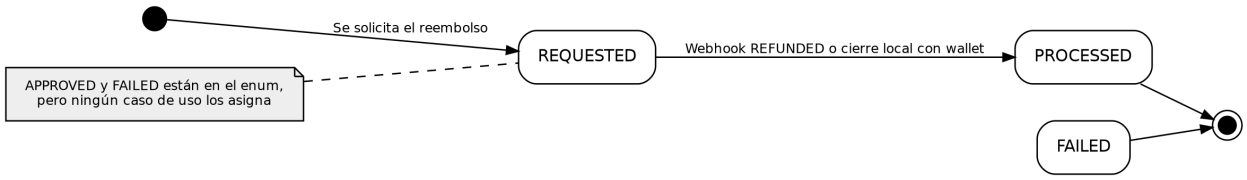


Tabla 11

Transiciones de la entidad Reembolso

Origen	Destino	Disparador	Condiciones (guards)	Clase y método
(nuevo)	REQUESTED	Se solicita el reembolso (usuario o cancelación del evento)	Sin método de transición en la entidad	RequestRefundUseCaseImpl (línea 70)
REQUESTED	PROCESSED	Webhook REFUNDED del gateway	Se construye un nuevo Refund con status PROCESSED y processedAt = now()	ProcessWebhookUseCaseImpl (línea 190)
REQUESTED	PROCESSED	Cierre local del reembolso (pago solo con wallet)	Se asume que corresponde a closeRefundLocally(); pendiente de confirmar	RequestRefundUseCaseImpl (línea 113)

RefundStatus.APPROVED y RefundStatus.FAILED no tienen ningún uso en el proyecto: están declarados en el enum, pero ningún caso de uso los asigna.

Relaciones entre entidades (causa y efecto)

Los cambios de estado de una entidad disparan cambios en otras. La Figura 11 resume las cadenas principales y la Tabla 12 indica la clase donde ocurre cada una.

Figura 11

Cadenas de causa y efecto entre las entidades del sistema

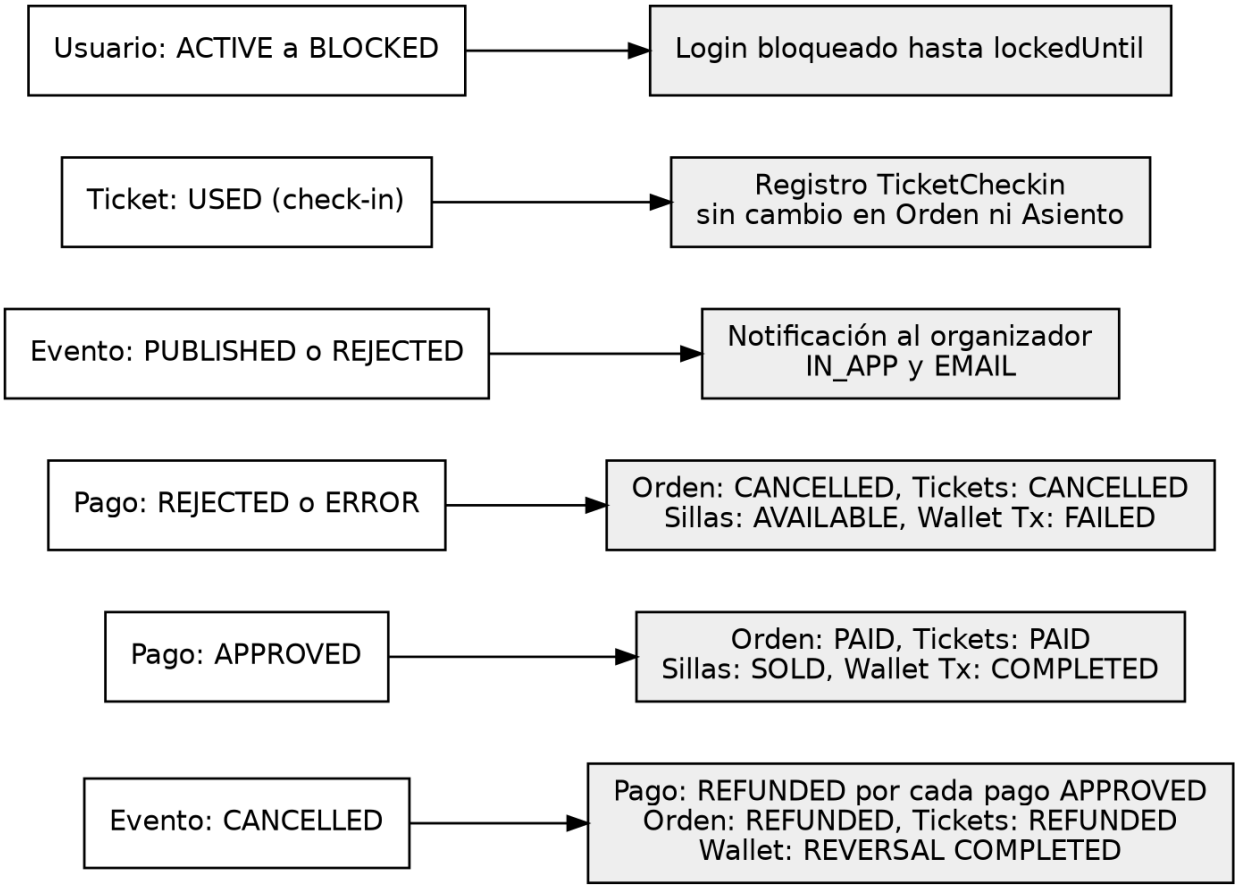


Tabla 12

Detalle de las relaciones de causa y efecto

Causa	Efecto	Clase donde ocurre
Pago (backend): PENDING → APPROVED (webhook APPROVED)	Orden: PAYMENT_PENDING → PAID; Tickets: PROCESSING → PAID; Sillas: RESERVED → SOLD	ProcessWebhookUseCaseImpl.execute() → ConfirmOrderPaymentUseCaseImpl.confirmOrderPayment() + SeatUseCase.updateSeatStatus()
Pago (backend): APPROVED creado directamente (solo wallet)	WalletTx: PENDING → COMPLETED; Orden: PAYMENT_PENDING → PAID; Tickets: PROCESSING → PAID; Sillas: RESERVED → SOLD	CreateWalletOnlyPaymentUseCase.execute() → ConfirmWalletPaymentUseCaseImpl + ConfirmOrderPaymentUseCaseImpl
Pago (backend): PENDING → REJECTED / ERROR (webhooks DECLINED, FAILED, CANCELLED)	WalletTx (si existía): PENDING → FAILED; Orden: PAYMENT_PENDING → CANCELLED; Tickets: PROCESSING → CANCELLED; Sillas: RESERVED → AVAILABLE	ProcessWebhookUseCaseImpl.execute() → ReleaseWalletPaymentUseCaseImpl + FailOrderPaymentUseCaseImpl.failOrderPayment() + SeatUseCase.updateSeatStatus()
Pago (backend): APPROVED → REFUNDED (webhook o pago solo con wallet)	Refund: REQUESTED → PROCESSED; Orden: PAID → REFUNDED; Tickets: PAID → REFUNDED; WalletTx: nueva transacción REVERSAL en COMPLETED (si había crédito wallet)	ProcessWebhookUseCaseImpl / RequestRefundUseCaseImpl.closeRefundLocally() → ConfirmRefundUseCaseImpl.confirmRefund() + ReverseWalletPaymentUseCaseImpl.execute()
Evento: PUBLISHED → CANCELLED	Reembolsos automáticos para todos los pagos APPROVED del evento (cada uno dispara la cadena de reembolso)	EventCancelledListener.handleEventStatusChanged() → RequestRefundUseCaseImpl.execute() por cada pago / for each payment
Evento: PENDING_REVIEW → PUBLISHED	Notificación al organizador (IN_APP y EMAIL): evento aprobado	EventStatusChangedListener (notification) → SendNotificationUseCase.execute()
Evento: PENDING_REVIEW → REJECTED	Notificación al organizador con el motivo del rechazo	EventStatusChangedListener → SendNotificationUseCase.execute()
Ticket: PAID → USED (check-in)	Se crea el registro TicketCheckin, sin cambio de estado en Orden ni Asiento	CheckinTicketUseCaseImpl.execute()
Usuario: ACTIVE → BLOCKED	Todos los intentos de login quedan bloqueados hasta que expire lockedUntil	AccountLockoutService.saveLockout() → blockUser()
Pago aprobado (cualquier tipo)	Se publica PaymentApprovedEvent para notificaciones	ProcessWebhookUseCaseImpl / CreateWalletOnlyPaymentUseCase → eventPublisher.publishEvent()
Wallet acreditada	Se publica WalletCreditedEvent para notificaciones	CreditWalletUseCaseImpl.execute() → eventPublisher.publishEvent()

Procesos automáticos

La Tabla 13 lista los schedulers, listeners y webhooks que cambian estados sin intervención directa del usuario.

Tabla 13

Procesos automáticos que cambian estados

Proceso	Frecuencia o disparo	Entidad afectada	Transición
EventAutoCompletionScheduler.completeExpiredEvents()	Diaria a las 00:05 (0 5 0 * * *) y una vez al arrancar (ApplicationReadyEvent)	Evento	PUBLISHED → COMPLETED para todos los eventos con finishDate < hoy
SeatReservationExpirationScheduler.releaseExpiredReservations()	Cada 10 segundos (*/10 * * * * *) y una vez al arrancar	Asiento	RESERVED → AVAILABLE para todos con reservedUntil < now
SecurityDataCleanupTask.unlockExpiredLockouts()	Cada minuto (0 * * * * *)	Usuario	BLOCKED → ACTIVE (limpia los AccountLockout expirados)
SecurityDataCleanupTask.cleanExpiredData()	Diaria a las 02:00 (0 0 2 * * *)	Sesiones y tokens (no son estados de negocio)	Elimina AuthSession y RefreshToken expirados o revocados; AccountActivation y RecoverPassword expirados o usados
ProcessWebhookUseCaseImpl.execute()	Invocado por el webhook HTTP del gateway	Pago, Orden, Tickets, Sillas, Wallet y Reembolso	Ver Tabla 12
EventCancelledListener.handleEventStatusChanged()	@EventListener sobre EventStatusChangedEvent (asíncrono, @Async)	Pago, Orden, Tickets, Wallet y Reembolso	Reembolso automático en cascada

Diferencias con la documentación existente

La Tabla 14 recoge las discrepancias entre lo que dicen los documentos del proyecto (Equipo CapySoft, s. f.-a, s. f.-b) y lo que hace el código (Equipo CapySoft, 2026).

Tabla 14

Discrepancias entre la documentación existente y el código

Discrepancia	Qué dice la documentación	Qué dice el código
Rol ATTENDEE en glosario y diagramas de clase	ATTENDEE como nombre del rol de usuario estándar	El código usa USER como DEFAULT_ROLE_CODE en RegisterUserUseCase y en los controladores
Estado IN_PROGRESS en el glosario del SRS	Lista IN_PROGRESS como posible estado de evento	EventStatus.java no contiene IN_PROGRESS; los 6 estados son DRAFT, PENDING_REVIEW, PUBLISHED, REJECTED, CANCELLED y COMPLETED
Manual técnico v1: 4 estados de evento	Documentaba DRAFT, PUBLISHED, CANCELLED y COMPLETED	El código tiene 6 estados desde que se añadió el flujo de revisión admin (changelog 079-add-event-review-flow.yaml)
Flujo de reactivación de cuenta	No documentado en el manual técnico ni en las historias de usuario del SRS	RequestReactivationUseCase y ConfirmReactivationUseCase están implementados y funcionales
Tiempo de bloqueo de cuenta	Manual técnico y manual de administrador documentan 30 minutos	LoginUseCase tiene LOCKOUT_MINUTES = 1 (static final, con un TODO para restaurar 30 en producción). Ningún archivo .java, .yml ni .properties define o lee app.security.lockout-minutes; el perfil application-k8s.yml no incluye esa propiedad
Transición Orden PAID → USED	El enum la declara como transición válida	Ningún caso de uso llama a order.markAsUsed(); la orden permanece en PAID después del check-in
Reembolso	El enum RefundStatus define 4 estados	Solo REQUESTED y PROCESSED se asignan; APPROVED y FAILED no tienen uso

Verificación y puntos abiertos

Los hallazgos de este documento se contrastaron con el repositorio mediante búsquedas en el código (grep y git grep) en la rama develop y en las 12 ramas remotas. A continuación se indica qué quedó verificado y qué sigue siendo una suposición.

Verificado

1. LOCKOUT_MINUTES vale 1 de forma fija en LoginUseCase y no se lee desde la configuración: no existe ningún @Value ni propiedad lockout-minutes en el proyecto.
2. El estado DELETED de la cuenta no es asignado por ningún caso de uso en ninguna de las 12 ramas remotas; solo aparece como guarda en LoginUseCase y DeactivateAccountUseCase.
3. Un webhook PROCESSING del fake-gateway se ignora con un log.warn y no provoca una transición inválida; además, la idempotencia por gatewayEventId descarta eventos repetidos.
4. Order.markAsUsed() no tiene ninguna llamada en el proyecto.
5. Los dos estados iniciales de Payment (PENDING y APPROVED) corresponden a las dos rutas de creación del pago.

Suposiciones pendientes

1. SeatService.updateSeatStatus() no valida el estado origen. Se asume que la validación se delega a los casos de uso de mayor nivel; no hay pruebas del módulo seat que confirmen el comportamiento esperado.
2. CancelOrderUseCaseImpl puede cancelar tickets en PENDING porque ticket.cancel() no valida el estado origen. Se asume que es intencional para la cancelación temprana desde una orden en DRAFT.
3. La asignación de RefundStatus.PROCESSED en RequestRefundUseCaseImpl (línea 113) se asume que corresponde al cierre local de reembolsos de pagos solo con wallet (closeRefundLocally()); falta confirmarlo.

Referencias

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Equipo CapySoft. (2026). *TuEvento* [Código fuente]. Repositorio del proyecto.

Equipo CapySoft. (s. f.-a). *Especificación de requisitos de software (SRS) de TuEvento* [Documento inédito]. Servicio Nacional de Aprendizaje (SENA).

Equipo CapySoft. (s. f.-b). *Manual técnico de TuEvento* [Documento inédito]. Servicio Nacional de Aprendizaje (SENA).

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML)* (Versión 2.5.1). https://www.omg.org/spec/UML/2.5.1